

Object-Oriented Programming

Classes and Objects

Ridwan Rismanto

What will we learn

- Basics of creating a class
 - Create attributes and methods (behaviour)
 - Basics of creating objects
 - Calling method
 - Manipulate attributes
- 

Case example: Banking information system

- We will create a banking information system program
- In the banking information system, one of them contains customer data.
- How do I manage customer data?
 - We are creating a class of customers (nasabah).
- This Nasabah Class has the following features:
 - Have the attributes of the customer's name and balance
 - Can display balance amount

How to Declare a Class

- Each class we create must be saved in a separate file
 - Classes declared with the keyword `public` must be stored with the same filename, with the extension `.java`
 - For example, the Nasabah's public class must be stored in the `Nasabah.java` file
 - REMEMBER! Lowercase case letters determine
- 

Attribution

- An attribute is a variable that is declared in a class
 - Attributes are used to store the state or traits of a class
 - For example, in our example this time, the Nasabah class has a name and balance attribute to store the name and balance of the class
- 

Method

- Method is also called a function
- A method is a behavior or "something that can be done" by the class
- Like functions, methods can also accept parameters, and can also have a return value
 - Method that doesn't have a return value → use "void"
- For example, in this example, the Nasabah class has a Balance display method that can display how much balance the customer has

How to Create an Object

- The class we've created won't be useful if we don't create an instance of it
 - Or in other words, create an object from that class
- The way to create an object is to use a new keyword
- For example, creating an object named nsb from the Customer class:

Customer nsb = new Customer();



Class Name

Object name

Class name again

Class Nasabah

- First of all, we create a Nasabah class
- This Nasabah class has the attributes of customer nama and saldo, and the method of tampilkanSaldo

```
public class Nasabah
{
    public String nama;
    public int saldo;

    public void tampilkanSaldo()
    {
        System.out.println("Saldo anda adalah " + saldo);
    }
}
```

Class Name

Name and
balance
attributes

DisplayBalance
Method/function

Inside of class Nasabah

Please observe the class Nasabah:

- Keyword **public** is an access modifier
 - Keyword **void** indicates that the method/function does not return a value
- 

Test unit

- If you notice, the Nasabah class does not have a main() method.
- So if we try to execute this class, it will be an error.
- It was deliberately created that way, because we are going to separate this main() method into another class, which we call a Test unit.
- We use test units as a program to test the classes that we have created before.
- For our test unit this time, we will create the **TestPerbankan** class as follows:

Class TestPerbankan

```
public class TestPerbankan
{
    public static void main(String[] args)
    {
        Nasabah nsb = new Nasabah();

        nsb.nama = "Bill Gates";
        nsb.saldo = 1000;

        nsb.tampilkanSaldo();
    }
}
```

Method main

Creating an nsb object
from the Nasabah class

Populate the name and
balance attributes of the
nsb object

Calling the Display Balance
method from the nsb
object

```
E:\Java\Perbankan>javac TestPerbankan.java
```

```
E:\Java\Perbankan>java TestPerbankan
Saldo anda adalah 1000
```

Inside class TestNasabah

Please observe the TestNasabah class:

- This class is an application because it has a main() method that will be executed first by the JVM
- Method main() is a special method because it is static
 - A static method allows us to call the method without having to create an object from that class
- In the main() method, we create an nsb object from the Nasabah class
- Then we fill in the name and balance attributes of the nsb object
- Then we call the tampilkanSaldo() method of the nsb object
- Methods on objects can be called by using a dot separator (.)
 - Example: nsb.tampilkanSaldo()

Compile multiple classes

- When compiling, we only need to compile classes that have the `main()` method, then the JVM will automatically search for other classes needed in the same directory/folder.
 - In our example, there are `Nasabah` and `TestPerbankan` classes, we just need to complete the `TestPerbankan` class.
 - Class `Nasabah` will be automatically compiled by JVM.
- 

Class Diagram

- A class diagram is a diagram used to illustrate the class schema on an object-oriented system
- In the diagram class there are:
 - Class Name
 - Attribute/property/variable class
 - Method/function

Example:

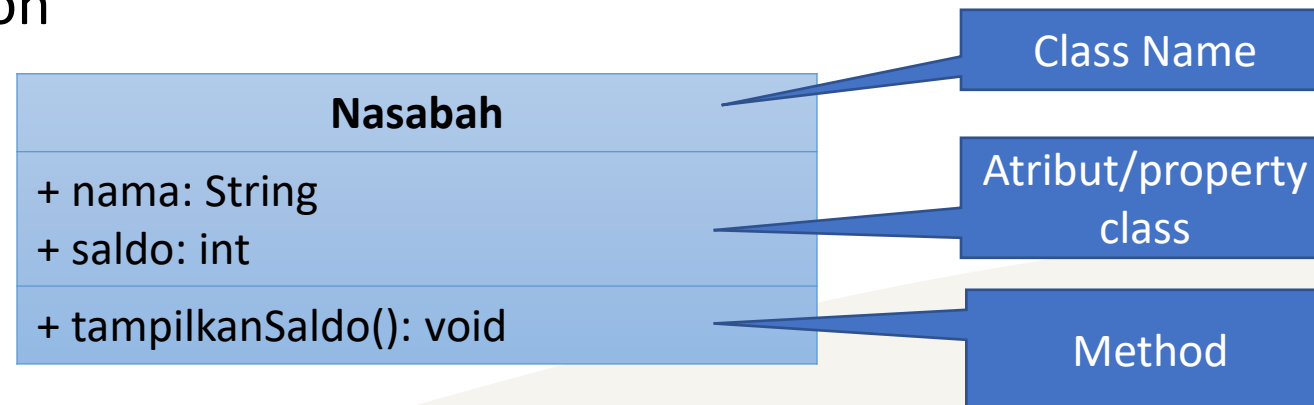


Diagram classes for Nasabah

Nasabah
+ nama: String + saldo: int
+ tampilkanSaldo(): void

Methods that have

- Next, we modify the Nasabah class, namely in the display method, we change the balance to accept a parameter in the form of a currency code (IDR/USD/JPY, etc...).

```
public class Nasabah
{
    public String nama;
    public int saldo;

    public void tampilkanSaldo(String mataUang)
    {
        System.out.println("Saldo anda adalah " + mataUang + saldo);
    }
}
```

Accepting currency
parameters with a
String type

Call the currency
parameter to appear
on the screen

TestPerbankan

- Next, we compile and run the TestPerbankan class again whether the changes we made to the Nasabah class worked as we expected
 - We enter the string "IDR" as a parameter in the display method

```
public class TestPerbankan
{
    public static void main(String[] args)
    {
        Nasabah nsb = new Nasabah();
        nsb.nama = "Bill Gates";
        nsb.saldo = 1000;

        nsb.tampilkanSaldo("IDR");
    }
}
```

Enter the string "IDR"
as the parameter

```
E:\Java\Perbankan>javac TestPerbankan.java
E:\Java\Perbankan>java TestPerbankan
Saldo anda adalah IDR1000
```

Class diagram Nasabah - updated

- Since we have added the currency parameter to the Display Balance method, now the Nasabah diagram class looks like this:

Nasabah
+ nama: String + saldo: int
+ tampilkanSaldo(mataUang: String): void

Create more than one object

- We can create more than one object of the same class
- We try to create two customer objects, namely nsb1 and nsb2, in the TestPerbankan class

```
public class TestPerbankan
{
    public static void main(String[] args)
    {
        Nasabah nsb1 = new Nasabah();
        Nasabah nsb2 = new Nasabah();

        nsb1.nama = "Bill Gates";
        nsb1.saldo = 1000;

        nsb2.nama = "Ridwan Rismanto";
        nsb2.saldo = 9999;

        nsb1.tampilkanSaldo("IDR");
        nsb2.tampilkanSaldo("IDR");
    }
}
```

```
E:\Java\Perbankan>java TestPerbankan
Saldo anda adalah IDR1000
Saldo anda adalah IDR9999
```

Class Modifications

- When we modify a class, the change will take effect immediately on all objects created from that class
- For example, we change the method of displaying the balance (saldo) in the Nasabah class to display the customer's name

```
public class Nasabah
{
    public String nama;
    public int saldo;

    public void tampilkanSaldo(String mataUang)
    {
        System.out.println("Saldo " + nama + " adalah " + mataUang + saldo);
    }
}
```

Display name
attributes

```
E:\Java\Perbankan>java TestPerbankan
Saldo Bill Gates adalah IDR1000
Saldo Ridwan Rismanto adalah IDR9999
```

Access Modifiers

- We declared the name, balance and method attributes of displayBalance() with the public access modifier, so that it can be accessed from other classes (i.e. the TestPerbankan class)
- Some frequently used access modifiers:
 - Private
 - It can only be accessed by methods in the same class
 - Public
 - Accessible by methods outside of the classroom
 - Protected
 - Only accessible to its child class

Nasabah class summary

- From our experiment just now, we can conclude that:
- Nasabah Class has name and balance attributes. This class also has a method for displaying the customer's name and balance, and also accepts currency parameters.

Nasabah
+ nama: String + saldo: int
+ tampilkanSaldo(mataUang: String): void

Question?